

What is Microsoft Foundry?

Open with the naming story (30 seconds):

Microsoft has renamed this platform three times in three years. Azure AI Studio in 2023, Azure AI Foundry at Ignite 2024, and now Microsoft Foundry in early 2026. The rename isn't cosmetic—it reflects a fundamental architectural shift underneath.

Explain the shift (1 minute):

The old model—Azure AI Studio—was backed by a Machine Learning Hub resource. You needed a mandatory Storage Account and Key Vault just to create a project. Three to four resources before you could do anything. The new model is a single lightweight Foundry Resource under CognitiveServices. Projects are child resources. No mandatory siblings. You go from zero to a running agent in minutes.

The resource provider changed from MachineLearningServices to CognitiveServices. That's significant—different APIs, different SDKs, different deployment model. If you're still on the old Hub model, you're on a dead architecture.

Land the resource hierarchy (1 minute)

There are now three levels.

Foundry Resource—IT admins own this. They set networking, encryption, model deployments.

Foundry Project—developers own this. They build agents, run evaluations, test.

Agent Application—when you publish, the agent becomes its own Azure resource with a dedicated endpoint and unique identity.

The key governance insight: developers cannot accidentally expose a public endpoint or disable encryption. Those decisions are locked at the resource level and inherited by all projects underneath.

Why "Microsoft" not "Azure" (30 seconds)

It now supports models from Anthropic, Meta, Mistral—not just Azure OpenAI. Microsoft is positioning this as the universal AI platform. The portal is still ai.azure.com, the resource types haven't changed—so operationally it's still Azure. The branding is aspirational.

Close with the maturity signal (20 seconds)

70,000 organizations are on this platform. Agent Service went GA in March 2026 with production SLAs and BAA for HIPAA. This is not a beta. The hard limits: 128 tools per agent, 10K files per thread, 100K messages per thread.

Transition to Slide 2:

So that's what Foundry is. Now let's look at what you actually build inside it - the three agent types.

Question: Classic vs New?

Answer: Always use New. Agent Service, multi-agent workflows, Memory, MCP, publishing to Teams - all New-only. Classic is the old Hub architecture. No reason to use it for new projects.

Agent Types & How They Work

Open with the framing (20 seconds):

Foundry gives you three ways to build an agent. They differ in how much control you want versus how much the platform handles for you. Think of it as a spectrum: no-code, low-code, full-code.

Prompt Agent (30 seconds)

Simplest type. Configuration only - no code. You write a system prompt, pick a model, attach tools, done. The model does all the reasoning. You control behavior through the prompt and tool selection. Use this for single-turn tasks - FAQ bots, lookups, simple retrieval. If the task is 'given this input, use these tools, return this output' - Prompt Agent. Limitation: it can't branch, loop, or pause for human approval within a single turn.

Workflow Agent (45 seconds)

Multi-step with a visual flow designer. Drag-and-drop nodes - LLM calls, tool calls, conditions, loops, parallel branches, human-in-the-loop approval gates. The flow itself is deterministic - you define the path. The LLM nodes within the flow are non-deterministic - the model reasons. But the overall structure is fixed and auditable.

The killer feature for regulated industries: human-in-the-loop nodes. The workflow pauses, waits for a human to approve or reject, then continues.

Example: claims adjudication — intake, validate eligibility, check fraud indicators, calculate payment, human approval if above threshold, process payment.

Hosted Agent (45 seconds)

Full code control. You write a Python or .NET application using any agent framework - Semantic Kernel, AutoGen, LangGraph, CrewAI, or completely custom. Deploy as a container to Microsoft-managed infrastructure. You set min/max replicas for auto-scaling. No portal designer - pure code.

When to use: complex custom reasoning that can't be expressed as a visual workflow. Multi-model orchestration. Custom state management. **Example:** fraud detection engine that runs proprietary scoring algorithms.

Caveat: It's in preview. No production SLA. Use for experimentation and non-critical workloads today.

Unified Orchestration Layer (30 seconds)

Represents the Foundry Agent Service - the runtime engine that executes all three agent types. It has the Reasoning loop, Tool dispatch, Thread management, Token management and Guardrail enforcement

Land the publishing model (1 minute)

Here's the important part that applies to ALL three types. When you publish any agent - Prompt, Workflow, or Hosted — it becomes an **Agent Application**. That's a separate Azure resource. Will cover it later in the Interaction Layer slide.

Connected Agents - quick mention (30 seconds)

Any agent can delegate to other agents. The parent sees connected agents as tools. Max nesting depth: 2 levels. The most powerful pattern: a Workflow Agent defines the deterministic flow, and at each step it delegates to a Prompt Agent that handles the reasoning. You get auditability of a workflow with flexibility of LLM reasoning at each node.

Transition to Slide 3

So those are the agent types. Now let's look at what the platform gives these agents out of the box - the services layer.

Question: Which type should we start with?

Answer: Prompt Agents for quick wins and POCs. Workflow Agents for anything with approval gates or multi-step processes. Hosted Agents only when you've outgrown what the other two can't express.

Question: Can we mix them?

Answer: Yes. That's the Connected Agents pattern. A Workflow Agent orchestrates, Prompt Agents execute individual steps. You're not locked into one type.

Platform Services

Open with the framing (15 seconds)

Now that you know the agent types, let's look at what the platform gives these agents out of the box. Six services—you don't build these, you configure them.

Tools (30 seconds)

Foundry ships 17 built-in tools across four categories: retrieval, integration, automation, and memory.

- File Search (retrieval)
- Code Interpreter (automation)
- Bing Grounding (retrieval)
- MCP connector (integration)
- Memory Search/Store (memory)
- Azure AI Search (retrieval)
- SharePoint (retrieval)
- OpenAPI (integration)
- Azure Functions (automation)

There are 1,400 more in the AI Tool catalog. Each agent can have up to 128 tools attached. But practically, more tools means worse tool selection by the model—10 to 20 per agent is the sweet spot.

The `tool_choice` parameter gives you deterministic control. Set it to 'auto' and the model decides which tool to call. Set it to a specific tool name and you force that tool. Use specific names for deterministic flows where you know exactly what should happen.

MCP (45 seconds)

Model Context Protocol—this is how agents talk to external systems. It's an open standard, originally created by Anthropic, adopted by Microsoft as a first-class protocol. GA—production ready.

An MCP server is a lightweight service that exposes tools the agent can call. You host it in your VNet on Azure Container Apps with internal-only ingress. Not exposed to the internet. The agent calls the MCP server, the MCP server calls the external system—Pega, Oracle, Snowflake, whatever.

Three approval modes per tool: 'always' means a human must approve every call. 'Never' means auto-approved—use only for validated read-only tools. 'Per-tool' lets you set it individually. 100-second timeout per call.

Authentication: Managed Identity preferred—no secrets to rotate. The agent never handles credentials. The platform injects them at call time.

Memory Service (30 seconds)

Preview—not for PHI until GA. This solves the goldfish problem. Without memory, agents forget everything between sessions.

Two memory types: User Profile Memory—facts about the user like preferences and history. Chat Summary Memory—compressed summaries of past conversations.

Three phases: Extraction—the LLM identifies key facts from a conversation. Consolidation—merges new facts with existing memories, deduplicates, resolves conflicts. Retrieval—before responding, the agent searches memories for relevant context.

Limits: 100 scopes per store, 10K memories per scope. A scope is typically a user ID.

Foundry IQ (30 seconds)

Microsoft's managed agentic retrieval. Instead of you building a RAG pipeline—chunk documents, embed, store in a vector DB, retrieve, rerank—Foundry IQ does it all.

'Agentic' means the retrieval system reasons about what to search for. It may reformulate the query, search multiple times, combine results, and decide when it has enough. Three effort levels: minimal, low, medium. 36% average quality improvement over standard vector search—that's Microsoft's benchmark.

Point it at Blob Storage, SharePoint, or uploaded files. It handles indexing, chunking, and retrieval.

AI Gateway (45 seconds)

This is Azure API Management with AI-specific policies. It sits between your agents and everything they call - model endpoints, tools, connectors etc. Five things it does:

One: token rate limiting per project. Prevents one runaway agent from consuming all your capacity.

Two: load balancing. Configure PTU as primary, PAYG as overflow. When PTU is exhausted, traffic automatically overflows. No code changes.

Three: semantic caching. If two requests are semantically similar - not just identical - return the cached response. Saves tokens, reduces latency.

Four: Content safety policies can be applied to all agents which provide fleet-wide enforcement. This is different from Foundry Guardrails, which is at the agent level.

Five: cost attribution. APIM adds custom dimensions - agent ID, project ID - to every request. App Insights stores the traces. KQL queries aggregate cost per agent, per project, per department. Zero code modification to your agents.

Cosmos DB (30 seconds)

Three roles: agent state persistence, chat history storage, and semantic cache backend. It uses DiskANN vector indexes for high-performance semantic search over chat history. BM25 for keyword search. Hybrid search via Reciprocal Rank Fusion combines both.

Practical example: 'find all conversations where a member discussed orthodontic coverage' - that's a semantic search over Cosmos DB chat history.

Transition to Slide 4

So those are the six platform services. Now the question leadership always asks: how do we govern all of this? That's next — governance and observability.

Question: *Do we need all six?*

Answer: *Tools and MCP are mandatory - that's how agents do anything useful. AI Gateway is strongly recommended for cost control and rate limiting. Cosmos DB is needed for chat history. Memory and Foundry IQ are optional but high-value. Start with Tools + MCP + Gateway + Cosmos, add Memory and IQ when you need them.*

Question: *What about semantic caching vs Foundry IQ - aren't they both about avoiding redundant calls?*

Answer: *Different problems. Semantic caching avoids calling the LLM again for similar questions - saves tokens and money. Foundry IQ improves the quality of retrieval before the LLM is called - better answers. One saves cost, the other improves quality. They work together.*

Governance & Observability

Open with the leadership question (15 seconds)

This is the slide that answers the question leadership always asks: 'How do we control this?' The answer is the Foundry Control Plane - a single pane of glass for your entire agent fleet.

Control Plane (45 seconds)

The Control Plane sits above all projects. It gives you fleet-level visibility - every agent across every project in a single view. Three capabilities:

Agent Registry—auto-discovers all agents in Foundry projects. For third-party agents running on other platforms, you manually register them. Once registered, same fleet view.

Lifecycle management—deploy, pause, kill, restart. These are control plane operations. You don't need access to the project itself to stop an agent.

Kill switch—revokes the agent's ability to respond. All in-flight requests complete, no new requests accepted. This is your emergency containment action. Target: under 15 minutes from detection to full stop.

One important architectural detail: token limits are enforced per project, not per agent. If you need to limit a specific agent's consumption, isolate it in its own project. Project isolation is your cost containment boundary.

Evaluations (45 seconds)

Three-stage evaluation lifecycle. Think of it as test-driven agent development.

Pre-build: before writing the agent, define what 'good' looks like. What are the failure modes? What are the safety boundaries? This creates your test suite before the implementation.

Pre-production: before deploying, run the evaluation suite. Quality metrics—coherence, fluency, relevance. RAG metrics—groundedness, completeness. Safety metrics—jailbreak resistance, PII leakage. If scores fall below thresholds, deployment is blocked. There's an Intelligent Gate that automates this decision.

Post-production: after deployment, continuously evaluate on sampled live traffic. Pick a percentage of real requests, run them through the evaluation pipeline, track scores over time. If groundedness drops from 0.92 to 0.85 over two weeks, something changed - you catch drift before users notice.

Red Teaming (30 seconds)

The AI Red Teaming Agent is itself an AI - it generates adversarial prompts designed to break your agents. 20-plus attack strategies: jailbreak, prompt injection, PII extraction, persona manipulation, multi-turn escalation.

You track Attack Success Rate per agent. Good target: below 5%. Above 10%, the agent is not ready for production. Run before every deployment, weekly on production agents, and after any prompt or model change.

Monitoring Dashboard (30 seconds)

Built-in Agent Monitoring Dashboard. No setup required—activates when you connect Application Insights. Per-agent: success rate, latency P50/P99, token consumption, error rates. Per-fleet: total active agents, anomaly detection that flags agents deviating from their baseline.

The daily operational question it answers: 'Are my agents healthy?' Click on an agent, drill down to individual traces—see the exact sequence of LLM calls, tool calls, and responses for a specific request.

Cost Attribution (30 seconds)

AI Gateway captures token metrics at the project level out of the box — every model call is logged with project identifiers and token counts. App Insights stores these traces. With additional gateway configuration, you can get per-agent granularity by emitting agent-level dimensions. KQL queries then let you slice cost by project, by agent, or by custom dimensions like department — using lookup tables you define. Zero code modification to your agents — all instrumentation happens at the gateway layer. You can add cost tracking to existing agents without redeploying them.

Alerting (30 seconds)

Built on Azure Monitor. Five key alert patterns:

Token spike—catches runaway agents.

Latency degradation—model overload or tool timeouts.

Error rate breach—something is broken.

Guardrail trigger surge—indicates an attack or misconfigured agent.

You track burn rate—how fast you're consuming your error budget. Alert when burn rate exceeds threshold, not just when the SLO is already breached. Early warning, not post-mortem.

Transition to Slide 5

So we can govern and observe the fleet. But who are these agents? How do they authenticate? That's identity and security — next.

Question: *What's the Intelligent Gate?*

Answer: *It's an automated decision engine in the deployment pipeline. Evaluation results come in, the gate checks against your thresholds, and it either passes the agent to production or blocks it. No human in the loop for the gate decision itself - you define the rules upfront.*

Question: *How is this different from just monitoring with App Insights?*

Answer: *App Insights gives you raw traces. The Control Plane gives you agent-aware context on top - it knows what an agent is, what its health baseline looks like, and can take action (pause, kill) directly. It's the difference between a log viewer and an operations console.*

Identity & Security

Open with the "why this matters" (20 seconds)

Agents are autonomous decision-makers that access sensitive data and take actions on behalf of humans. They need their own identity—not a repurposed service principal, not a shared managed identity. Microsoft built a purpose-built identity type for this: Entra Agent ID.

Three identity types (1 minute)

Three identity types in the architecture, each for a different actor:

Entra ID—for humans. Your claims processors, developers, admins. SSO, MFA, Conditional Access. Standard enterprise identity you already know.

Entra Agent ID—for agents. Purpose-built. Auto-provisioned when an agent is published. Scoped tokens—the agent only accesses what it needs. Conditional Access policies apply to agents too, same as humans. This is not a repurposed service principal. It understands the agent lifecycle: created, published, active, revoked.

Managed Identity—for resource-to-resource authentication. Foundry Resource to Key Vault, Agent Service to AI Search. No credentials stored anywhere. Automatic.

The key insight (30 seconds)

Before Entra Agent ID, agents used service principals—tools designed for applications, not autonomous decision-makers. A service principal has no concept of 'this identity makes decisions that affect humans' or 'this identity should be revocable in 15 minutes.' Entra Agent ID solves that. Short-lived tokens, automatic refresh, instant revocation via the kill switch.

When you publish an agent, it gets a new Entra Agent ID—different from the dev identity. The dev identity has broad project access. The production identity has only what the agent needs. You must explicitly assign RBAC to the production identity.

Conditional Access for agents (1 minute)

This is the part that surprises people. Same Conditional Access policy engine your security team already uses for humans—now applied to agents.

Network location: 'This agent can only operate from VNet X.' If the agent configuration is stolen and run from outside the approved network, access denied.

Risk-based: 'If this agent's risk score exceeds the threshold, block it.' Risk score comes from behavioral signals — anomalous token consumption, unusual tool call patterns, guardrail trigger frequency.

Compliance state: 'This agent must have guardrails enabled and CMK active.' If someone disables guardrails, Conditional Access blocks the agent from operating until they're re-enabled.

Critical detail: these policies are evaluated on every request — not just at login time. Continuous access evaluation for agents.

Zero Trust (45 seconds)

Traditional zero trust says assume the network is compromised. For agents, the principle is: assume the prompt is compromised. Someone has injected malicious instructions into the agent's input. Design for that.

Verify explicitly — every agent call authenticated, every tool call carries a scoped token. The MCP server verifies the token before responding. No anonymous agent operations.

Least privilege — RBAC per agent, not per project. allowed_tools whitelist per MCP connection—the agent can only call specific tools, not everything on the server. Data-level scoping—the MCP server returns only the fields the agent needs.

Assume breach — guardrails scan at four points: input, tool call, tool response, output. Red teaming runs continuously. Kill switch is always one click away. 15-minute containment SLA.

RBAC layers — quick (20 seconds)

Four RBAC layers:

Resource Owner for IT admins — they control infrastructure.

Project Developer for builders — they create agents.

Agent Application User for end users — they can talk to the agent but can't modify it.

Control Plane Contributor for the platform team — they can pause or kill agents across the fleet.

Transition to Slide 6

So agents have identity, they're governed by zero trust. Now how do they actually talk to external systems? That's integration protocols — MCP and A2A.

Question: *What about agents on other platforms - Pega, Salesforce?*

Answer: *Entra Agent ID applies to Foundry agents only. External platform agents use their own identity systems. When they communicate via A2A protocol, authentication is handled at the protocol level — key-based, managed OAuth, or custom OAuth.*

Question: *Is Conditional Access for agents GA?*

Answer: *It's new. Validate policy enforcement with your specific Entra ID configuration. The policy engine is GA, but agent-specific signals like risk scoring based on behavioral patterns are still maturing.*

Integration Protocols

Open with the core question (15 seconds)

Your agents need to talk to external systems—Pega, Oracle, Snowflake, Salesforce. Foundry gives you two protocols for this. The question is: which one, when?

MCP — the workhorse (1.5 minutes)

MCP—Model Context Protocol. GA, production ready. This is your primary integration pattern.

The mental model is simple: the agent is the master, the external system is the tool. One-way relationship. Agent calls system, system responds. The agent is in control.

An MCP server is a lightweight stateless HTTP service. It exposes a manifest—here are the tools I offer—and handles invocations. Agent says 'create a case in Pega with these parameters,' MCP server receives the request, calls the Pega REST API, transforms the response, returns structured data to the agent.

Hosting: Azure Container Apps with internal-only ingress, inside your VNet. Not exposed to the internet. NSGs restrict traffic — only the Agent Subnet can reach the MCP Subnet.

Authentication is two-hop. Agent to MCP server: Managed Identity preferred—no secrets to rotate. The platform injects credentials at call time. MCP server to external system: the server uses its own credentials stored in Key Vault. The agent never sees either set of credentials.

Enterprise strategy: Pega and Salesforce already have community MCP servers—extend them, don't rebuild from scratch. Oracle, Snowflake, Dataiku—no existing servers, build custom ones wrapping their REST APIs. Typical effort: 4 to 6 weeks per module for Oracle, 3 to 4 weeks for Snowflake. SharePoint and Blob Storage — Foundry has built-in tools, no MCP server needed.

A2A — the future (45 seconds)

A2A—Agent-to-Agent. Preview, not production ready. Originally created by Google, adopted by Microsoft.

Completely different mental model. Both sides are agents—neither is in charge. They exchange messages, negotiate, collaborate as peers.

Use case: Pega has its own agentic AI that understands claims processing deeply. Rather than rebuilding that intelligence in Foundry, use A2A to let the Foundry agent collaborate with the Pega agent. Each brings its own expertise.

Status: Preview. Do not use for production workloads. Do not route PHI through A2A until GA with BAA coverage. Plan for it in Phase 2 or 3 of your roadmap. For now, use MCP to talk to these systems.

Decision tree (45 seconds)

Here's the decision framework. Four questions in order:

One: does the external system have its own AI agents? If yes — plan for A2A in the future, use MCP for now.

Two: does an MCP server already exist for that system? If yes — extend it. Pega, Salesforce — saves weeks of development.

Three: does it have REST APIs? If yes — build a custom MCP server. This is the most common path for enterprise systems.

Four: none of the above? Function Calling with custom code. This is the fallback for legacy systems with non-standard interfaces.

There's also OpenAPI—you just point the agent at a Swagger spec and it calls the endpoints directly. Simplest option. But you lose MCP's governance features: approval modes, `allowed_tools` whitelist, AI Gateway governance. Use for simple, low-risk integrations where spinning up a full MCP server is overkill.

Transition to Slide 7

So agents can talk to external systems. Now the question is: how do users and applications talk to the agents? That's the interaction layer.

Question: *Why not just use OpenAPI for everything?*

Answer: *You lose governance. MCP gives you approval modes — human must approve before the agent calls a tool. It gives you `allowed_tools` whitelists — the agent can only call specific tools, not everything. And the AI Gateway can govern MCP calls centrally. OpenAPI is fire-and-forget — fine for low-risk, but not for systems that handle PHI or financial data.*

Question: *When will A2A be GA?*

Answer: *No announced date. Microsoft adopted it in preview. Plan your architecture to support it in Phase 2/3, but don't depend on it for Phase 1. MCP covers 90% of integration needs today.*

Interaction Layer

Open with the publishing concept (30 seconds)

You've built an agent, tested it in your project. Now what? You publish it. Publishing creates an Agent Application—a separate Azure resource. This is the step that moves your agent from 'works in my project' to 'ready for others to use.'

What you get on publish: a dedicated stable REST endpoint—the URL never changes even when you deploy new versions. A unique Entra Agent ID—separate from the project identity. Independent RBAC scope—grant access to one agent without exposing others. Start/stop lifecycle management. And because it's an ARM resource, Azure Policy can govern it.

The identity gotcha — land this hard (30 seconds)

The number one gotcha teams hit when publishing for the first time: identity changes. In dev, all agents share the project's Entra Agent ID. When you publish, the agent gets its own unique identity. Permissions do NOT transfer automatically. You must reassign RBAC to the new identity for every resource the agent accesses. If you skip this, tool calls that work in dev fail in production with auth errors. Build this into your deployment checklist.

Teams / M365 (45 seconds)

Channel one: Microsoft 365 and Teams. For internal users. You publish the agent to the Teams agent store.

Two scopes:

Individual — no admin approval needed, good for pilots and small teams.

Organization — admin approval required, appears in 'Built by your org' section, production deployment.

Users find the agent in Teams, chat naturally. Zero new UI to build. Your claims processors, service reps — they already live in Teams. Put the agent where they already are.

Limitations to know: no Private Link support for Teams/Bot Service. No streaming — responses come back complete, not token-by-token. File uploads and image generation don't work in M365, only in Teams. For PHI-heavy interactions, use the custom UI channel where Private Link is enforced end-to-end.

Responses API (45 seconds)

Channel two: Responses API. For customer-facing experiences. Your web or mobile app calls the Agent Application endpoint directly. You control the entire UI — branding, authentication, conversation UX, disclaimers, escalation paths. The intelligence is Foundry's; the experience is yours.

Authentication is two layers: your app authenticates to the Agent Application using Entra tokens. Your end users authenticate to your app using whatever auth you already have — member portal login, OAuth, whatever. The agent never sees end-user credentials directly.

Streaming is supported — token-by-token for responsive UX. You can pass custom metadata — user context, session info — to personalize agent behavior. This is your channel for member portals, provider portals, any external-facing experience.

Voice Live (30 seconds)

Channel three: Voice Live. Preview. Real-time speech-to-speech. Fully managed — audio capture, speech recognition, agent reasoning, speech synthesis all handled by the platform. No audio model deployment on your side.

Use case: phone-based member support. Member calls, speaks naturally, agent responds with voice. Potential to replace or augment IVR systems. But it's preview — no BAA coverage for voice yet. Plan for Phase 3 when it reaches GA.

Close with the key principle (15 seconds)

The same agent can be published to multiple channels simultaneously. Build once, distribute everywhere. And guardrails — content safety, grounding, PII protection — apply identically across all channels. Configure once at the agent level, enforced everywhere regardless of how the user reaches the agent.

Transition to Slide 8

So we've covered what agents are, what services they use, how they're governed, how they authenticate, how they integrate, and how users reach them. Last piece: how do we lock down the infrastructure underneath all of this?

Question: *Can one agent be on Teams AND Responses API?*

Answer: Yes. Same agent, multiple channels simultaneously. The agent doesn't know or care which channel the request came from. Guardrails apply the same way regardless.

Question: *What about Agent 365?*

Answer: Newer capability for deeper M365 integration. Gives agents access to organizational context - Work IQ, user's calendar, emails. Evaluate for Phase 2. It's the difference between an agent that answers questions and an agent that understands your work context.

Infrastructure & Security

Open with the leadership framing (15 seconds)

Last slide. Everything we've discussed—agents, services, governance, identity, integrations, channels—all of it sits on top of infrastructure. This is where your security and compliance teams need to be comfortable. Three pillars: networking, data protection, compliance.

Networking (1.5 minutes)

Two modes

Basic—Microsoft-managed network, no VNet integration. Use for dev and prototyping.

Standard—your VNet, your private endpoints, full network isolation. Use for production. For anything touching sensitive data, Standard is non-negotiable.

Standard setup gives you four subnets:

Agent Subnet—the Foundry Agent Service behind a private endpoint.

MCP Subnet—Azure Container Apps with internal-only ingress, where your MCP servers live.

Data Subnet—AI Search, Cosmos DB, Storage, Key Vault, all behind private endpoints.

Gateway Subnet—API Management.

NSGs restrict traffic between subnets. Only the Agent Subnet can reach the MCP Subnet. Only the MCP Subnet can reach the Data Subnet. Key Vault has the most restrictive rules. No public endpoints anywhere. All traffic stays within the VNet or traverses private endpoints.

One limitation to own: Teams and Bot Service do NOT support Private Link. Internal agents published to Teams communicate over Microsoft's managed network—encrypted, SOC 2 compliant, but not your private network.

Design principle: Teams agents are workflow assistants, not data display terminals. For PHI-heavy interactions, use the Responses API channel where Private Link is enforced end-to-end.

Data Protection (1 minute)

Three layers of encryption.

Data at rest: Customer-Managed Keys via Azure Key Vault. You control the keys, not Microsoft.

Data in transit: TLS 1.2+—automatic, nothing to configure.

Data in processing: Azure Confidential Computing where available.

CMK setup: Key Vault in the same region as your Foundry Resource, soft delete and purge protection enabled, RSA key minimum 2048 bits, managed identity with Key Vault Crypto User role.

One critical detail: CMK cannot be reverted once enabled. You can't go back to Microsoft-managed keys. Plan this decision carefully.

Data privacy guarantees: your prompts and completions are not shared with other customers. Not used to train or improve Microsoft models. Data stays in the region you deploy to. Abuse monitoring opt-out is available for approved customers — needed when prompts contain PHI that shouldn't be logged.

Compliance (1.5 minutes)

Three products working together.

Azure Policy—preventive. Defines and enforces rules. 'All Foundry Resources must use CMK.' 'All model deployments must have guardrails.' Blocks non-compliant deployments before they happen.

Microsoft Defender—detective. Runtime threat detection. Anomalous agent behavior, data exfiltration attempts, unusual token consumption patterns.

Microsoft Purview —protective. DLP policies for PHI, sensitivity labels, data classification, compliance reporting. Tells you where your sensitive data is and who's accessing it.

Land the GA vs Preview table (30 seconds)

The rule for regulated industries: if it's Preview and it touches PHI, don't use it. Or get written BAA confirmation from Microsoft.

GA and safe for PHI: Agent Service, Azure OpenAI, AI Search, Cosmos DB, Storage, API Management, Key Vault. These are your production building blocks.

Preview — do NOT use for PHI: Memory Service, A2A protocol, Voice Live, agent-level guardrails, Hosted Agents. Use these for non-sensitive workloads. Plan to adopt them when they reach GA

Incident response — quick mention (30 seconds)

If something goes wrong: Detection via Defender alerts and monitoring dashboard. Containment via kill switch—target under 15 minutes. Assessment via OpenTelemetry traces and KQL—target under 4 hours. If PHI breach confirmed, HIPAA requires notification to HHS within 60 days. The architecture supports this entire lifecycle.

Close the presentation (15 seconds):

That's Microsoft Foundry end to end. An agent factory with built-in governance, purpose-built identity, enterprise networking, and a clear line between what's production-ready.

Question: *What about HIPAA specifically?*

Answer: *BAA is available. The HIPAA checklist: CMK for encryption, Standard VNet for network isolation, Entra Agent ID plus RBAC for access control, App Insights plus OpenTelemetry for audit trails, Purview for DLP, Defender for threat detection. All GA. The gap is agent-level guardrails — those are Preview. Supplement with GA model-level guardrails as your primary enforcement.*

Question: *What's the PHI minimization strategy?*

Answer: *Three layers. Tool level: allowed_tools whitelist — agent can only call specific tools. Agent level: RBAC per identity — agent only accesses what it needs. Data level: MCP servers return only the fields required. Minimum tools per agent, minimum data per tool, minimum access per identity.*

Question: *What happens if an agent goes rogue?*

Answer: *Kill switch via Control Plane—stops the agent immediately. AI Gateway rate limit to zero. Revoke the Entra Agent ID token. All in-flight requests complete, no new requests accepted. Target: under 15 minutes from detection to full containment*